Coursework 2 – A Box-pushing Game

Introduction

This is the second COMP1038 Coursework. It is worth **45% of the module mark**. It requires you to write an interactive program in C that simulates a box-pushing game. The deadline for this exercise is **18:00 on Monday 22nd of December 2025**.

Read the entire document before beginning the exercise.

If you have any questions about this exercise, please ask in the Q&A forum on Moodle, after a tutorial session, or during the advertised office hours. Do not post your program or parts of your program to Moodle as you are not allowed to share your coursework programs with other students. If any questions require this exercise to be clarified, then this document will be updated, and everyone will be notified via Moodle.

Submission

You must submit a single C source code file containing all your code for this exercise. This file must be called **boxpushing.c** and must not require any other files outside of the standard C headers. The first line of the file should be a comment which contains your student ID number, username, and full name, of the form:

```
// 6512345 zy12345 Joe Blogs
```

The file must compile without warnings or errors when I use the command

```
gcc -Wall -lm boxpushing.c -o boxpushing
```

This command will be run on X2Go. If it does not compile, for any reason, then you will lose all the marks for testing (common reasons in the past have been submitting a file with the wrong filename or developing your solution on your personal computer without having tested it on X2Go). If the file compiles but has warnings, then you will lose some marks for not correcting the warnings.

Before you submit your source code file, you must complete the **Coursework submission coversheet** on Moodle.

The completed source code file should be uploaded to the Coursework 1 Submission

link on the COMP1038 Moodle page. You may submit as many times as you wish before the deadline (the last submission before the deadline will be used). After the deadline has passed, if you have already submitted your exercise, then you will not be able to submit again. If you have not already submitted, then you will be allowed to submit **once**.

Late submissions: Late submissions will lose 5 percentage points **per hour**, rounded up to the next whole hour. This is to better represent the large benefit a small amount of extra time can give at the end of a programming exercise. No late submissions will be accepted more than 24 hours after the exercise deadline. If you have extenuating circumstances, you should file them before the deadline.

Plagiarism

You should complete this coursework on your own. Anyone suspected of plagiarism will be investigated and punished in accordance with the university policy on plagiarism (see your student handbook and the University Quality Manual). This may include a mark of zero for this coursework.

You should write the source code required for this assignment yourself. If you use code from other sources (books, web pages, etc), you should use comments to acknowledge this (and marks will be heavily adjusted down accordingly). *The only exception to this is the example programs given in lectures and tutorials; you may use them, with or without modification, without penalty.*

You must not copy or share source code with other students. You must not work together on your solution. You can informally talk about higher-level ideas but not to a level of detail that would allow you all to create the same source code.

Remember, it is quite easy for experienced lecturers to spot plagiarism in source code. If you are having problems, you should ask questions rather than plagiarize. If you are not able to complete the exercise, then you should still submit your incomplete program as that will still get you some of the marks for the parts you have done (but make sure your incomplete solution compiles and partially runs!).

Task

Your task is to develop a C program that simulates a box-pushing game. The game takes place on a 12×12 grid, as illustrated in the figure below. A robot is represented by the symbol '@', the boxes are represented by 'O', the walls are represented by '#', and the empty spaces are filled by '.'.

```
############
#..o..o...o#
#......o...#
###o..o.o..#
#..o@..o.oo#
#o#..o.....#
#o...oo..o.#
#.oo...o.oo#
#....o.....#
#..o...o#o.#
#....o.....#
############
```

The robot can move around the map according to the user's command. There are four possible directions for each move (^ for up, v for down, < for left, > for right). As the robot attempts to move, if there are any boxes (o) in the way, the robot will also attempt to push those boxes. However, if this action would cause the robot or a box to move into a wall (#), nothing moves instead, including the robot. The initial positions of these are shown on the above map.

When the program starts, it should display the interface shown below and then wait for user input.

```
z2017155@csubuntu:~/Desktop/CW2$ ./boxpushing
############
#..o..o...o#
#......o...#
###o..o.o..#
#..o@..o.oo#
#o#..o.....#
#o...oo..o.#
#.oo...o.oo#
#....o.....#
#..o...o#o.#
#....o.....#
############
Enter command (q to quit):
```

As the user enters command, the robot should attempt the given sequence of moves, pushing around the boxes as the following examples:

```
z2017155@csubuntu:~/Desktop/CW2$ ./boxpushing
############
#..o..o...o#
#......o...#
###o..o.o..#
#..o@..o.oo#
#o#..o.....#
#o...oo..o.#
#.oo...o.oo#
#....o.....#
```

```
#..o...o#o.#
#....o.....#
############
Enter command (q to quit): >
############
#..o..o...o#
#......o...#
###o..o.o..#
#..o.@.o.oo#
#o#..o.....#
#o...oo..o.#
#.oo...o.oo#
#....o.....#
#..o...o#o.#
#....o.....#
############
Enter command (q to quit): vv
############
#..o..o...o#
#......o...#
###o..o.o..#
#..o...o.oo#
#o#........#
#o...@o..o.#
#.oo.o.o.oo#
#....o.....#
#..o.o.o#o.#
#....o.....#
############
Enter command (q to quit): vv
############
#..o..o...o#
#......o...#
###o..o.o..#
#..o...o.oo#
#o#........#
#o...@o..o.#
#.oo.o.o.oo#
#....o.....#
#..o.o.o#o.#
#....o.....#
############
Enter command (q to quit): >v>>v
############
#..o..o...o#
#......o...#
###o..o.o..#
#..o...o.oo#
#o#........#
#o.....o.o.#
#.oo.o..ooo#
#....o.@...#
#..o.o.o#o.#
#....o.....#
############
Enter command (q to quit): >v>x
Invalid command, enter again: <> vv
Invalid command, enter again: >q
Invalid command, enter again: q
z2017155@csubuntu:~/Desktop/CW2$
```

You should use at least one data structure in your program, such as a linked list. An additional advanced requirement is that the program should terminate normally once all boxes on the map cannot be moved.

Marking

The marking scheme will be as follows:

- *Tests (85%):* Your program should correctly implement all task requirements. A number of tests will be run against your program to verify that your program produces correct outputs. The specific test cases will remain confidential, but general examples may include:
    - Does the program work with typical valid input?
    - Does the program correctly deal with invalid input?
    - Does the program work with special input data, such as extremely long string?

  As noted in the submission section, if your program does not compile, **then you will lose all testing marks**. If the format of outputs is incorrect, then you will lose up to **50% of testing marks.**
- *Language and code formatting (15%):* Your program should correctly use at least one data structure. Your program should correctly use functions, rather than putting large amount of code in the main() function. Your program should be correctly formatted and easy to understand by a competent C programmer. This includes, but is not limited to, indentation, bracketing, variable/function naming, and use of comments.

Appendix

A testing tool for you to check the output format is provided on Moodle. You need to download all files to your local machine and then transfer them to the same folder as your CW2 program (e.g. boxpushing.c) in your X2Go account. If the files are downloaded as a compressed archive, you should extract (decompress) them before transferring them to X2Go.

Open a terminal and type "chmod u+x *.*".

Type "./markone.sh" to start testing.

If you see the messages of "Output matches" in green, it means that your program passes the tests.

END